

SKILL - 17

Name: Hansika Reddy Gunapati

Rollno: 2520030237

Section: 8

Experiment No. 17 (Q17)

Title: Implementation of Active Job Listing and Foreground Job Control (jobs and fg) in a Custom Shell

Aim:

To implement how to list active jobs, display status information, retrieve job IDs, format output, update state information and test job listings. To design how to identify a target job, transfer terminal control, resume processes, wait for completion, update job state and test foreground switching.

Objective:

Part 1 – Listing Active Jobs

- To list every active background job maintained by the shell.
- To display status information (Running, Stopped, Done) for each job.
- To retrieve the job ID assigned when a job was launched.
- To format the output in the conventional "[job] state command" style.
- To update state information by polling process status without blocking.
- To test job listings across running, stopped and completed jobs.

Part 2 – Foreground Job Control

- To identify the target job from the job ID passed to fg.
- To transfer terminal control to the job's process group.
- To resume a process that was previously stopped.
- To wait for completion, or for the job to stop again.
- To update the job's state once control returns to the shell.
- To test foreground switching on valid and invalid job IDs.

Theory:

1. Job Control in a Shell

A job is a command line launched by the shell, either in the foreground or with & in the background. Interactive shells keep a job table so that background and stopped work can be listed, resumed, or brought back to the foreground without losing track of it. Each job is identified by a small integer job ID rather than its raw process ID, which is easier for a user to type.

2. Process Groups and a Job's Own pgid

Every job is placed in its own process group with `setpgid()`, called both in the child right after `fork()` and in the parent to avoid a race. Grouping a job's processes together means a signal such as `SIGSTOP`, `SIGCONT` or `SIGINT` can be delivered to the whole job at once with `kill(-pgid, signal)`, and the terminal can be handed to the group as a single unit.

3. The Job Table and Job IDs

The job table is an array of Job structures, each holding a job ID, the process ID, the process group ID, the original command text and a state. A new job is appended with the next unused job ID; `addJob()` and `findJobById()` are the only functions that touch this table directly, which keeps the bookkeeping in one place.

```
typedef struct {
    int jobId; pid_t pid; pid_t pgid;
    char command[64]; enum JobState state;
} Job;
```

4. Non-blocking Status Polling with `waitpid()`

`waitpid(pid, &status, WNOHANG | WUNTRACED | WCONTINUED)` checks a single job without suspending the shell. `WNOHANG` returns immediately if nothing has changed, `WUNTRACED` reports a job that has just stopped, and `WCONTINUED` reports one that has just resumed. This is what lets "jobs" refresh every job's state before printing it.

5. Formatting the jobs Output

Each active job is printed as one line: the job ID in brackets, a + marking the most recently referenced job, the state name, and the original command text. Once a job's status becomes exited or signalled it is reported as Done exactly once and then dropped from the table, so the listing never grows without bound.

6. Terminal Control and `tcsetpgrp()`

Only one process group at a time is the terminal's foreground group, and only that group receives terminal-generated signals such as `SIGINT` or `SIGTSTP`. `tcsetpgrp(STDIN_FILENO, pgid)` moves that ownership to a job's group before it runs in the foreground, and back to the shell's own group afterwards. The call only makes sense when standard input is an actual controlling terminal, so `isatty()` guards it.

7. Resuming Stopped Jobs with `SIGCONT`

A job that was stopped (for example by Ctrl-Z, or here by an explicit `SIGSTOP` used to test the state machine) is not running and will not respond to being waited on until it is resumed. `kill(-pgid, SIGCONT)` wakes every process in the group; `fg` does this before it starts waiting, and the job's state is updated to Running immediately since `SIGCONT` normally succeeds.

8. Waiting for Foreground Completion

Once a job owns the terminal and is running, `fg` calls `waitpid(pid, &status, WUNTRACED)` and blocks. Three outcomes are possible: the job exits or is killed by a signal, in which case it is marked Done or Terminated; or the job stops itself again, in which case it is marked Stopped and control is simply returned to the shell so the job can be resumed later.

9. Removing Finished Jobs

listActiveJobs() first refreshes every job's state, then prints the ones that are still active, and finally compacts the table by dropping any job marked Done after printing a one-line notice for it. This mirrors how a real shell reports "Done" once and does not keep repeating it on every subsequent jobs call.

Algorithm:

Part 1 – Job Listing

1. Start the program.
2. Launch a command in the background: fork a child, place it in its own process group, and record it in the job table with a new job ID.
3. To list jobs, poll every tracked process with a non-blocking waitpid() call.
4. Update each job's state from the returned wait status (running, stopped, exited, continued).
5. Print one line per active job as [id] state command.
6. Report and remove jobs whose state has become Done.
7. Repeat the listing as jobs are added, stopped, or completed.
8. Stop the program.

Part 2 – Foreground Switching

1. Start the program.
2. Read the job ID requested by fg.
3. Look up the job ID in the job table; report an error if it does not exist or has already finished.
4. If the job is stopped, send SIGCONT to its process group and mark it running.
5. Transfer the controlling terminal to the job's process group with tcsetpgrp(), when a controlling terminal is available.
6. Block on waitpid() for that job until it exits, is terminated, or stops again.
7. Return the terminal to the shell's own process group.
8. Update and report the job's final state (Done, Terminated, or Stopped).
9. Stop the program.

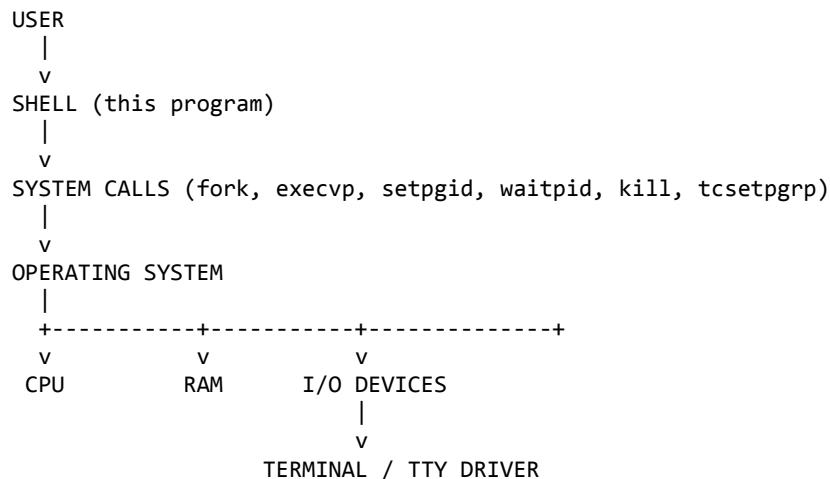
Functions Used:

Function	Purpose
addJob()	Creates a new job table entry with a fresh job ID
findJobById()	Retrieves a job by the ID the user typed
updateJobStates()	Polls every tracked pid without blocking and refreshes its state
formatJobLine()	Formats one job as the shell's jobs output line
listActiveJobs()	Refreshes, prints, and compacts the job table
fgJob()	Identifies, resumes, foregrounds and waits for one job
launchBackground()	Forks and execs a command as a new background job
fork()	Creates the child process that becomes a job
setpgid()	Places a job in its own process group
execvp()	Replaces the child image with the job's command
waitpid()	Polls (WNOHANG) or blocks (WUNTRACED) on a job's status
kill()	Sends SIGSTOP/SIGCONT to a job's whole process group
tcsetpgrp()	Moves the controlling terminal to or from a job's group
isatty()	Checks whether standard input is a real controlling terminal
getpgrp()	Reads the shell's own process group ID at start-up
strerror()	Converts errno into a readable error message

Hardware and Operating System Relationship

Hardware consists of physical components such as the CPU, RAM, storage devices, keyboard, mouse and display. The Operating System acts as an interface between application software and hardware. It manages processes, memory, files, devices and system resources.

Relationship



How OS Interacts with Hardware

- **Process Management** – The OS creates a process group per job and schedules each job's process independently of the shell.
- **Memory Management** – The OS manages the job table, the per-process memory image created by `execvp()`, and the kernel's process-control blocks that track each job's state.
- **File Management** – The OS keeps the same descriptor table across `fork()`, so a job's open files are unaffected by being moved between background and foreground.
- **Device Management** – The OS tracks which process group currently owns the controlling terminal and delivers terminal-generated signals only to that group.
- **Security** – The OS restricts `tcsetpgrp()` and signal delivery to processes that share the calling session, so unrelated processes cannot hijack terminal control.

Observation

Two background jobs (`sleep 2 &` and `sleep 6 &`) were launched and registered with job IDs 1 and 2. The first jobs call correctly reported both as Running. Sending `SIGSTOP` to job 2 and calling jobs again showed its state updated to Stopped purely from the non-blocking `waitpid()` poll, with job 1 unaffected. Calling `fg 2` resumed the stopped job with `SIGCONT`, attempted to transfer terminal control (skipped safely since this run has no controlling terminal), waited for it to finish, and reported it as Done. A further jobs call showed no active jobs and reported both jobs as Done exactly once before removing them. Requesting `fg` on a non-existent job ID (99) was correctly rejected with an error message instead of crashing.

Result

Job listing and foreground control were successfully implemented, including background job registration under separate process groups, non-blocking state polling with `waitpid()`, conventional jobs-style output formatting, resuming stopped jobs with `SIGCONT`, terminal control transfer with `tcsetpgrp()`, blocking wait for foreground completion, and validation of invalid job IDs.

Design / Flow

Job Listing Flow

```

Launch command with &
|
v
fork() child, setpgid(child, child)
|
v
addJob(pid, pgid, cmd) -> new job ID, state = Running
|
v
jobs requested
|
v
updateJobStates(): waitpid(pid, WNOHANG|WUNTRACED|WCONTINUED) for every job
|
+-----+-----+-----+
|         v         v         v         v
no change  Stopped  Running  Done
|
v
print [id] state command for every job still active
|
v
report and drop jobs whose state is Done

```

Foreground Switching Flow

```

fg <jobId>
|
v
findJobById(jobId) --not found--> print error, return
| found
v
state == Stopped? --yes--> kill(-pgid, SIGCONT), state = Running
|
v
isatty(stdin)? --yes--> tcsetpgrp(stdin, pgid) --no--> skip, note it
|
v
waitpid(pid, WUNTRACED) (blocks)
|
+-----+-----+
|         v         v         v
exited    signalled  stopped again
|         |         |
v         v         v
state=Done state=Terminated state=Stopped
|
v
tcsetpgrp(stdin, shellPgid) -- return terminal to the shell

```

CODE-

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_JOBS 20
#define CMD_LEN 64

/* ===== Job state and structure ===== */
enum JobState { JOB_RUNNING, JOB_STOPPED, JOB_DONE };

typedef struct {
    int      jobId;
    pid_t    pid;
    pid_t    pgid;
    char      command[CMD_LEN];
    enum JobState state;
} Job;

static Job jobTable[MAX_JOBS];
static int jobCount = 0;
static int nextJobId = 1;
static pid_t shellPgid;

static const char *stateName(enum JobState s)
{
    switch (s) {
        case JOB_RUNNING: return "Running";
        case JOB_STOPPED: return "Stopped";
        case JOB_DONE:    return "Done";
    }
    return "?";
}

/* ===== Part 1: Listing Active Jobs ===== */

/* Adds a newly launched background process to the job table */
int addJob(pid_t pid, pid_t pgid, const char *cmdLine)
{
    if (jobCount >= MAX_JOBS) {
        printf(" job table full, cannot track pid %d\n", pid);
        return -1;
    }
    Job *j = &jobTable[jobCount++];
    j->jobId = nextJobId++;
    j->pid = pid;
    j->pgid = pgid;
    strncpy(j->command, cmdLine, CMD_LEN - 1);
    j->command[CMD_LEN - 1] = '\0';
    j->state = JOB_RUNNING;
    printf("[%d] %d\n", j->jobId, j->pid);
    return j->jobId;
}
```

```

}

/* Retrieves a job by its job id */
Job *findJobById(int jobId)
{
    for (int i = 0; i < jobCount; i++)
        if (jobTable[i].jobId == jobId)
            return &jobTable[i];
    return NULL;
}

/* Polls every tracked pid without blocking and updates its status.
   This is how the state column shown by "jobs" stays accurate. */
void updateJobStates(void)
{
    for (int i = 0; i < jobCount; i++) {
        Job *j = &jobTable[i];
        if (j->state == JOB_DONE) continue;

        int status;
        pid_t r = waitpid(j->pid, &status, WNOHANG | WUNTRACED | WCONTINUED);
        if (r == 0) continue; /* no change */
        if (r < 0) { j->state = JOB_DONE; continue; }

        if (WIFEXITED(status) || WIFSIGNALED(status))
            j->state = JOB_DONE;
        else if (WIFSTOPPED(status))
            j->state = JOB_STOPPED;
        else if (WIFCONTINUED(status))
            j->state = JOB_RUNNING;
    }
}

/* Formats one job line the way a shell's "jobs" builtin would */
void formatJobLine(const Job *j, int isMostRecent)
{
    printf("[%d]c %-8s %s\n",
           j->jobId, isMostRecent ? '+' : ' ',
           stateName(j->state), j->command);
}

/* Lists every job that has not finished, then removes finished ones */
void listActiveJobs(void)
{
    updateJobStates();

    int shown = 0;
    for (int i = 0; i < jobCount; i++) {
        if (jobTable[i].state == JOB_DONE) continue;
        formatJobLine(&jobTable[i], i == jobCount - 1);
        shown++;
    }
    if (shown == 0)
        printf(" no active jobs\n");

    /* compact the table: drop finished jobs, print a notice for each */
    int w = 0;
    for (int i = 0; i < jobCount; i++) {
        if (jobTable[i].state == JOB_DONE) {
            printf("[%d] Done %s\n", jobTable[i].jobId, jobTable[i].command);
            continue;
        }
    }
}

```

```

    }
    jobTable[w++] = jobTable[i];
}
jobCount = w;
}

/* ===== Part 2: Foreground Job Control (fg) ===== */

/* Brings a background/stopped job into the foreground and waits for it */
int fgJob(int jobId)
{
    /* Identify target job */
    Job *j = findJobById(jobId);
    if (j == NULL) {
        printf(" fg: %d: no such job\n", jobId);
        return -1;
    }
    if (j->state == JOB_DONE) {
        printf(" fg: %d: job has already finished\n", jobId);
        return -1;
    }

    printf(" bringing job [%d] (%s) to the foreground\n", j->jobId, j->command);

    /* Transfer terminal control to the job's process group.
       Only meaningful when stdin is an interactive terminal; in a
       non-interactive run there is no controlling tty to hand over. */
    if (isatty(STDIN_FILENO)) {
        if (tcsetpgrp(STDIN_FILENO, j->pgid) < 0)
            printf(" tcsetpgrp failed: %s\n", strerror(errno));
    } else {
        printf(" (no controlling terminal in this session - skipping tcsetpgrp)\n");
    }

    /* Resume the process if it was stopped */
    if (j->state == JOB_STOPPED) {
        if (kill(-(j->pgid), SIGCONT) < 0)
            printf(" kill(SIGCONT) failed: %s\n", strerror(errno));
        j->state = JOB_RUNNING;
    }

    /* Wait for completion (or for it to stop again) */
    int status;
    pid_t r = waitpid(j->pid, &status, WUNTRACED);

    /* Give the terminal back to the shell */
    if (isatty(STDIN_FILENO))
        tcsetpgrp(STDIN_FILENO, shellPgid);

    if (r < 0) {
        printf(" waitpid failed: %s\n", strerror(errno));
        return -1;
    }

    if (WIFEXITED(status)) {
        printf(" [%d]+ Done %s (exit %d)\n",
              j->jobId, j->command, WEXITSTATUS(status));
        j->state = JOB_DONE;
    } else if (WIFSIGNALED(status)) {
        printf(" [%d]+ Terminated %s (signal %d)\n",
              j->jobId, j->command, WTERMSIG(status));
    }
}

```



```

        j->state = JOB_DONE;
    } else if (WIFSTOPPED(status)) {
        printf(" [%d]+ Stopped    %s\n", j->jobId, j->command);
        j->state = JOB_STOPPED;
    }
    return 0;
}

/* ===== Launching background jobs for the tests ===== */

pid_t launchBackground(const char *cmdLine, char *const argv[])
{
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return -1; }

    if (pid == 0) {
        /* Child: its own process group so job control can target it */
        setpgid(0, 0);
        execvp(argv[0], argv);
        fprintf(stderr, "%s: command not found\n", argv[0]);
        _exit(127);
    }

    /* Parent: also sets the child's group (avoids a race with execvp) */
    setpgid(pid, pid);
    return pid;
}

/* ===== Test drivers ===== */

void testJobListing(void)
{
    printf("\n=== Part 1: Testing Job Listing ===\n");

    char *argv1[] = { "sleep", "2", NULL };
    pid_t p1 = launchBackground("sleep 2 &", argv1);
    addJob(p1, p1, "sleep 2 &");

    char *argv2[] = { "sleep", "6", NULL };
    pid_t p2 = launchBackground("sleep 6 &", argv2);
    addJob(p2, p2, "sleep 6 &");

    printf("\n jobs\n");
    listActiveJobs();

    printf("\n (stopping job 2 to test state tracking)\n");
    kill(p2, SIGSTOP);
    usleep(300000);

    printf("\n jobs\n");
    listActiveJobs();
}

void testForegroundSwitching(void)
{
    printf("\n=== Part 2: Testing Foreground Switching ===\n");

    /* job 2 (sleep 6) is currently STOPPED from the previous test */
    fgJob(2);

    printf("\n jobs\n");
}

```

```
        listActiveJobs();

        printf("\n fg 99   (invalid job id)\n");
        fgJob(99);
    }

/* ===== Main ===== */
int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0);
    shellPgid = getpgrp();

    testJobListing();
    testForegroundSwitching();

    printf("\nAll jobs processed. Final table:\n");
    listActiveJobs();
    return 0;
}
```

Compilation and Execution

1. Compile

```
gcc question17.c -o question17
```

2. Execute

```
./question17
```

Sample Output

```

teja@hasini:~/os_lab$ ./question17

=== Part 1: Testing Job Listing ===
[1] 199
[2] 201

jobs
[1]    Running    sleep 2 &
[2]+  Running    sleep 6 &

(stopping job 2 to test state tracking)

jobs
[1]    Running    sleep 2 &
[2]+  Stopped     sleep 6 &

=== Part 2: Testing Foreground Switching ===
fg 2
bringing job [2] (sleep 6 &) to the foreground
(no controlling terminal in this session - skipping tcsetpgrp)
[2]+  Done          sleep 6 & (exit 0)

jobs
no active jobs
[1]    Done          sleep 2 &
[2]    Done          sleep 6 &

fg 99  (invalid job id)
fg: 99: no such job

All jobs processed. Final table:
no active jobs
teja@hasini:~/os_lab$

```

Conclusion

This experiment demonstrates job control in a custom shell, including registering background commands as jobs under their own process groups, listing active jobs with states kept current through non-blocking `waitpid()` polling, and formatting that listing in the conventional jobs style. It also demonstrates bringing a job to the foreground with `fg`, covering resumption of stopped jobs with `SIGCONT`, transfer of terminal control with `tcsetpgrp()`, blocking until the job finishes or stops again, and rejection of an invalid job ID. Together, these illustrate important Operating System concepts such as process groups, session and terminal ownership, signal delivery scoped to a group, and the state transitions (Running, Stopped, Done) that a shell must track for every job it manages.